

A scanner gives you findings. A pentester gives you a plan.

websec-validator reads a repository the way a senior pentester reads it on day one, then hands your AI coding agent a plan: the supported attack surface, which leads need review, and how to test them. No model inside; the core pass needs no server or running app.

THE PROBLEM

AI agents now write the code. Nobody briefs them on what to attack. Static scanners answer “which patterns match?” and bury the answer under volume. A pentest answers “what matters, and how would I prove it?” and happens once a year, if at all.

```
A REAL COUNT, FROM THIS PROJECT'S OWN CHANGELOG (0.17.0, BUG-311)
findings-ledger.json    total: 163          # what the scanners said
cluster_summary        distinct issues: 13  # what a reviewer actually has to fix
```

Same fixture, same run. The **163** is not wrong, and every one is kept for the machine consumers. But a human, or an agent, has to be told they are **13 problems**, where they live, and what would confirm each one. That handoff is the product.

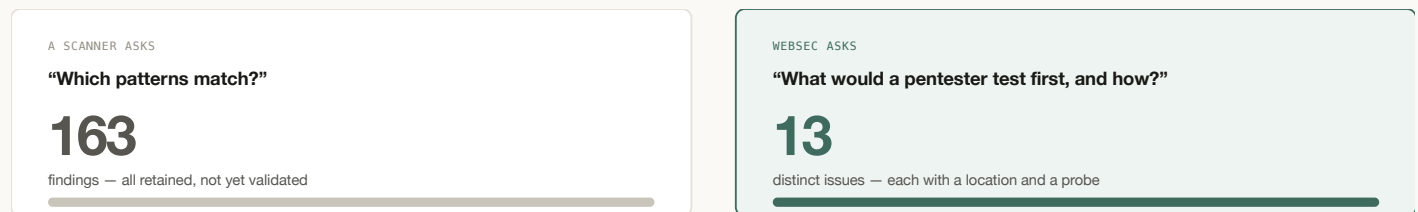


FIG. 1 — THE SAME LEDGER, UNDER TWO DIFFERENT QUESTIONS. THE SECOND ONE IS WHAT A PENTEST REPORT CONTAINS.

THE METHOD

code in

Read supported source. Map visible routes, auth decisions, tenant boundaries, sinks, secrets and dependency. Run the installed static scanners, de-duplicate them, and rank what is left by whether it is reachable and exploited in the wild.

artifacts out

A fact file, a findings ledger, a SARIF report, a probe library tailored to what was found, and a briefing for the agent that will do the fixing. Then an exit code that says whether the picture is *complete*.

The rule that surprises people: there is no model in the tool. Analysis is reproducible with the same inputs and tooling; run metadata and dated advice can vary. The model sits on the *other* side of the briefing, with a human beside it.

A tool that never says “incomplete” is a tool you cannot trust when it says “clean.”

THE DESIGN CONSTRAINT EVERYTHING ELSE FOLLOWS FROM

EVERY AMBIGUITY ROUNDS TOWARD “INCOMPLETE”

Most security tools have two outputs: findings, or a green check. This one has a third, and it is what makes the other two believable.

```
WHAT THIS TOOL SAID ABOUT A DELIBERATELY VULNERABLE ELIXIR APP, UNTIL SEPTEMBER 2026

# raw SQL interpolation · System.cmd injection · a hardcoded credential · no auth plug
unsupported: []  gaps: []
REQUESTED CHECKS COMPLETED — 0 files read
```

A suffix list had been absorbed into another, leaving the “language I cannot read” branch reachable for one language. The fix does not make the tool read Elixir. It makes it **say** it did not: the banner leads with `NO ANALYZABLE SOURCE`, and `--require-analyzed` turns that into exit 3.

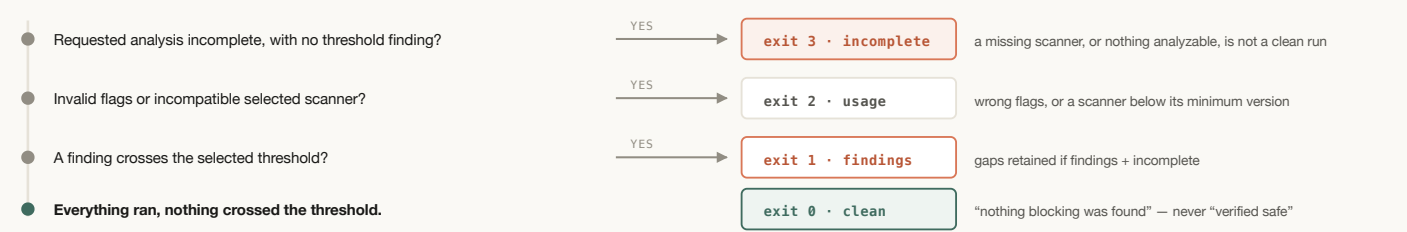


FIG. 2 — WITH `--require-complete` AND `--fail-on`. USAGE IS CHECKED FIRST; FINDINGS + INCOMPLETE EXITS 1, WITH BOTH RECORDED. UNGATED EXIT 0 IS NOT A COMPLETENESS VERDICT.

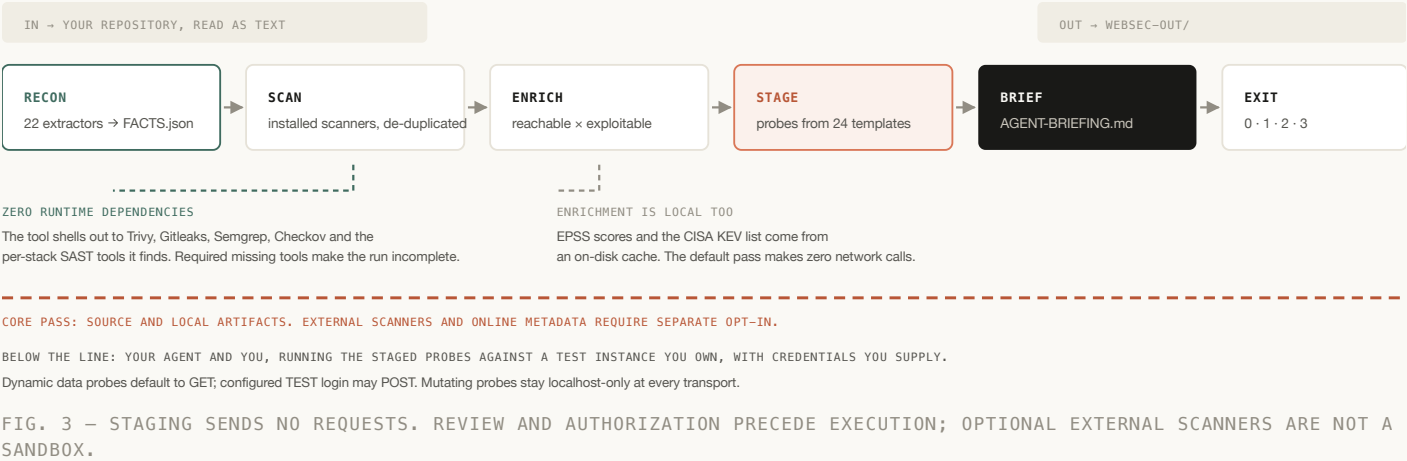
NINE THINGS THE TOOL WILL NOT DO, AND WHERE THAT IS ENFORCED

REFUSAL	WHY, AND WHERE
say “compliant”	The word appears in no output, and a test asserts that, so it cannot come back by accident. tests/test_attest.py
claim complete protection	<code>protection_complete</code> is always false: static analysis cannot see a running system. docs-canonical/SECURITY.md
fill an evidence gap	The audit projection has 12 control rows; 5 carry an empty evidence list, with the reason. attest.py
count “unknown” as “false”	The old rule scored every unmatched finding as a false positive. Relabelling against pinned source left 21 reviewed labels and 35 findings still unknown , excluded from both sides. calibration.json
optimize its own score	Anything fitted from labels uses a strictly proper rule, Brier or log score, never accuracy or F1, which reward confident wrongness. It is printed, never optimized. calibration.SCORING_RULE
run a scanner unasked	TruffleHog verifies secrets against live services, so it runs only when named. scanners.py
read a cell too thin to trust	A per-class probability needs at least five samples. One class reaches that today; the rest fall back a tier, and the output names which. calibration.MIN_N
guess reachability	A vulnerable dependency is tagged <code>imported</code> , <code>no-import-found</code> or <code>n/a</code> . It never rounds <code>n/a</code> to safe. enrichment.py
claim a benchmark it has not run	The competitor comparison exists as a written protocol. Until it runs, no head-to-head claim is made. BENCHMARKS.md

THE ONE PLACE IT FAILS OPEN, ON PURPOSE

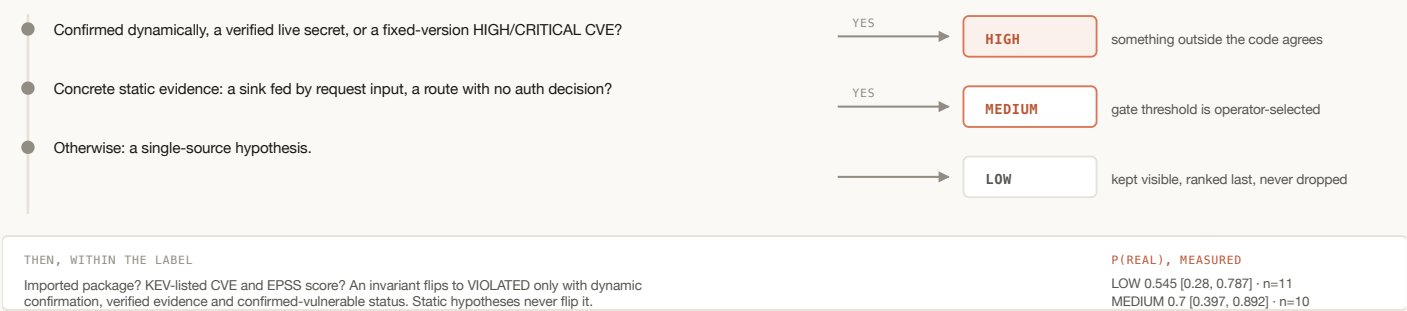
The in-editor hook **blocks on findings or completed-but-incomplete analysis**, including selected source it could not read. Only unexpected internal crashes fail open, loudly: the edit was not checked. Client-side settings remain editable, so production enforcement belongs in required CI checks, not this convenience hook.

ONE REPOSITORY, START TO BRIEFING



HOW A FINDING EARNS ITS SEVERITY

Severity is a rule, not a score. The same evidence yields the same label every run, and the rule is written in the methodology before it is written in the code.



WHAT LANDS IN WEBSEC-OUT/

ARTIFACT	WHAT IT IS
AGENT-BRIEFING.md	The product. Detected surface, the access-control map, targeting, findings, the method, and the staged probe list, written for the agent that will act on it.
FACTS.json	The full structured recon: every fact the 22 extractors produced, schema-versioned, with a coverage block that says what was skipped and why.
findings-ledger.json	The traceable ledger. Each finding carries its evidence chain, CWE / ASVS / OWASP-API citation, remediation, and a calibrated probability with its interval and sample size. REPORT.md is the same ledger for humans.
results.sarif	SARIF 2.1.0, always written. GitHub code scanning, GitLab, Azure DevOps and the VS Code SARIF viewer read it as-is.
probes/	Scripts staged from the 24 templates, pre-filled with the endpoints the recon found, for a human to review and run against a test instance.

TWENTY-TWO EXTRACTORS, SEVENTEEN SINK CLASSES, ONE FACT FILE

Each extractor answers one question a pentester asks on day one, and writes the answer into `FACTS.json` as data, never prose.

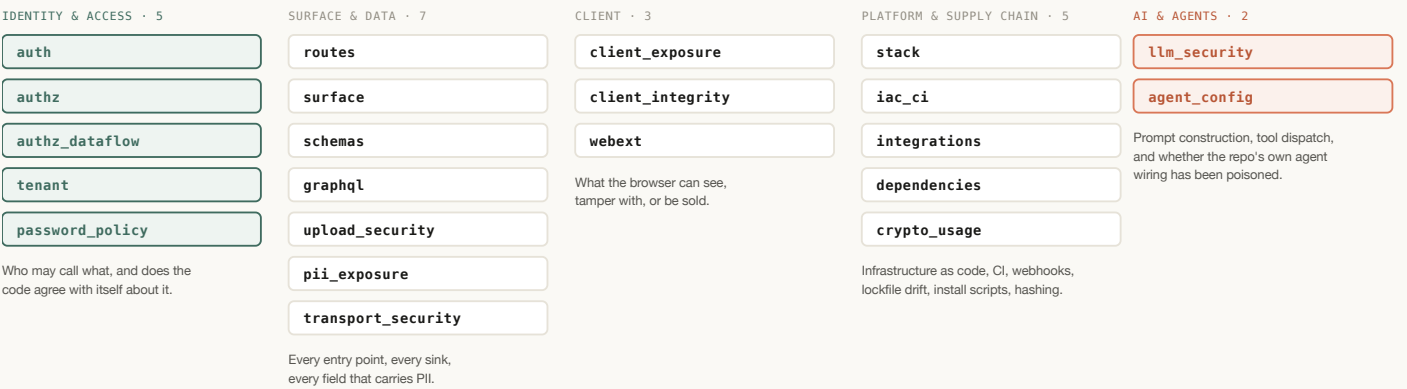
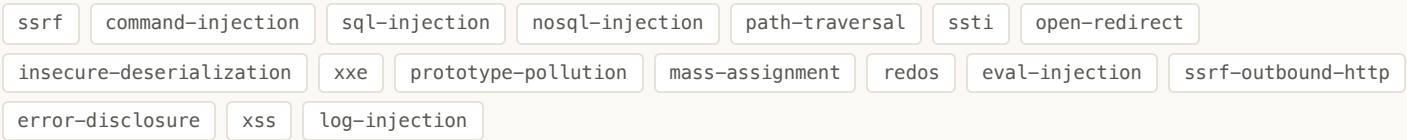


FIG. 5 — THE 22 EXTRACTORS. GROUPS ARE EDITORIAL; THE REGISTRY IS FLAT. EACH IS ISOLATED: ONE THAT CRASHES RECORDS AN ERROR IN COVERAGE AND THE RUN CONTINUES.

THE SEVENTEEN SINKS: REQUEST INPUT REACHING SOMETHING DANGEROUS, TRACED PER ROUTE



Why these twenty-two

Not designed at a whiteboard. Three rows exist because dogfooding on a large LLM-agent monorepo, verified by a 15-agent adversarial pass, exposed two classes the route-and-auth model could not see. Four more grew because a real authenticated pen test found its two Criticals and two Highs in files the recon never parsed: AWS-CDK auth config, AppSync GraphQL schema, VTL resolvers. The walker reads those now.

Mapped to what a reviewer will ask about

Every finding carries a CWE, an ASVS 4.0.3 reference and, where they apply, the OWASP API and OWASP LLM Top 10 entries. The methodology publishes the map as a 41-row table; the findings module cites 77 distinct CWEs. Two limits are stated up front: regex over CDK TypeScript is not an AST and can be evaded by aliasing, and man-in-the-browser can never be a confident static catch, so it ships as LOW.

THE SCANNERS IT SHELLS OUT TO, AND THE VERSION EACH ONE MUST BE

Presence on PATH is not compatibility, so each entry carries the lowest version whose CLI matches the invocation the tool builds, and `doctor` reports it.

SCANNER	CATEGORY · MINIMUM	PER STACK	RUNS ONLY WHEN THE LANGUAGE IS PRESENT
Trivy	vulnerabilities, secrets, misconfig · 0.38	Bandit	Python · 1.7
GitLeaks	secrets in history · 8.0; plus a working-tree pass	gosec	Go · 2.0
Semgrep/OpenGrep	SAST, with two shipped rules · 1.0	Brakeman	Ruby · 4.0
Checkov	infrastructure as code · 2.0	TruffleHog	live secret verification · opt-in only, never implicit
OSV-Scanner	dependency advisories · 2.0		
Prowler	cloud posture · detect only		

SIX PUBLIC REPOSITORIES, 116 LEADS, ZERO VERIFIED CRITICAL ISSUES

Dated source-only review: 14 September 2026. A tool that reported a critical in every public project it touched would be a tool that mislabels. The review ran source-only against pinned revisions, executed nothing, sent nothing upstream, and wrote down what it could not conclude.



FIG. 6 — ONE DOT PER LEAD. COUNTS ARE DETECTOR OUTPUT AT A PINNED REVISION; THE REVIEW SAYS SO ON ITS FIRST LINE.

116 leads emitted across six repositories, none executed, nothing sent upstream DOCS/SECURITY-REVIEW/PUBLIC-REPOSITORY-REVIEW.MD	0 verified critical issues. Written as the headline, not buried in a footnote SAME DOCUMENT, FIRST PARAGRAPH	3 → 10 false-positive classes adjudicated with source links, each turned into paired regression tests TESTS/TEST_PUBLIC_PRECISION.PY
---	---	---

What the false positives taught

- An archive error helper that formats ZIP messages was read as an **authentication decoder** because its strings said “unauthorized.”
- A local file stream piped to stdout was read as **HTTP file serving** because `createReadStream` alone counted as serving.
- A numeric GitHub expression in a workflow was read as **script injection**.

Each fix ships with its opposite: the test that makes the tool quieter is paired with one that proves the real case, the JWT decoded without verification, the stream piped to a response, still fires. The review also records its own gap: `@angular/core` was not recognized in the framework inventory.

The corpus, pinned

Dated v0.14.0 proof, not a v0.20.0 rerun or vulnerability recall. Three deliberately vulnerable apps at fixed commits, retrieved 2026-09-12: VAmPI `f16052d`, NodeGoat `c5cb68a`, DVGA `a961308`. The proof harness passes **10/10** surface checks with Noir present and **8/10** without it, with 7 truth labels left unknown. Both numbers are committed as JSON with the detector revision hash. DocGuard binds their validation-record statements to JSON fields; this brief uses separate fact tests and a PDF freshness manifest.

In September 2026 those 56 corpus findings were relabelled one at a time against the pinned source, each with a note and the revision it was read at. That produced **21 reviewed labels** and a measured probability per severity, replacing a prior that had been guessed. The other **35 stay unknown** and are counted on neither side.

And one real pentest: an authenticated engagement whose two Criticals and two Highs sat in the managed-cloud boundary the recon did not yet parse. Those findings became extractor rows, and two of the retest lessons corrected the tool’s own earlier output.

THE SUITE IS GUARDED THE WAY THE TOOL GUARDS YOU

Five checks gate the main branch. Two of them exist because of a specific, recorded failure.

TESTS · PY3.11 · PY3.12 · PY3.13

Install editable, byte-compile the package, smoke the CLI entry point, run the suite.

THREE REQUIRED CHECKS

HERMETICITY · HOSTILE GIT CONFIG

A global hooksPath, gpgsign, excludesFile and a renamed default branch. The suite must still pass, and sentinel hooks outside the temp repos must checksum clean.

SUITE-INTEGRITY

Derived test floor · 60 s ceiling · required-check names cross-checked against the jobs this run emits · unit tests for the bot that merges without a human.

WHY THE FLOOR HAS NO NUMBER IN IT

A stored MIN_TESTS was bumped 1241 → 1243 by two branches at once. Both merged cleanly, because the values matched, and the floor ended two below the real suite: two tests deletable unnoticed. So the baseline is now counted from the base commit at CI time, with the unittest loader, in a separate process per tree. A module that fails to import is a hard failure, not a count of one.

WHY SIXTY SECONDS

The suite runs in about ten. The known failure in this class was a 7.6x blowup that stayed green throughout. Sixty catches that shape and tolerates a slow runner.

FIG. 7 — THE GATE STACK. THE NAMES IN REQUIRED-CHECKS.JSON ARE THE SINGLE SOURCE OF TRUTH; A REQUIRED CHECK THAT NEVER RUNS WOULD BLOCK EVERY PR FOREVER.

1,713

tests across 87 files, stdlib only, no public network, at this snapshot

TESTS/ · COUNTED BY THE UNITTEST LOADER

0

runtime dependencies. The tool shells out to scanners and never imports them

PYPROJECT.TOML · DEPENDENCIES = []

5

required checks, cross-verified against the jobs the run actually emits

.GITHUB/REQUIRED-CHECKS.JSON

Docs are checked like code

DocGuard runs with 27 of its 29 validators on, pinned explicitly rather than inherited. Four canonical documents are review-gated. Five sentences in the validation record are bound to JSON pointers inside the committed proof artifacts, so a number in prose cannot differ from the number in the file. Every conscious deviation from those documents is logged in a drift file with its reason.

Publication has separate safeguards. Selected facts are checked by test_explained_brief.py ; a hash manifest binds the HTML, PDF and renderer script. CI rejects a stale pair. DocGuard checks marked Markdown sections, not PDF rendering. Prose accuracy and all eight rendered pages still need human review.

A release train that stops for a human

Every Monday a workflow proposes a version bump as a pull request and stops. A tag is cut only when a commit whose subject starts with release: lands on main, never from a version diff. Publishing to PyPI uses Trusted Publishing over OIDC, with no long-lived token, and refuses if the tag does not match the package version. The triage bot that labels and merges bot PRs never checks out the PR's code, and its classifier has its own unit tests in the required checks.

EVERY FIX CITES ITS BUG, AND EVERY QUIETING TEST HAS A LOUD TWIN

A field report from 2026-09-18 listed nine problems. They became 50 tests in one file, and the fixes carry the bug number as a comment at the line that changed, in the CLI, the scanner adapters, the findings module, the coverage contract and the dynamic phase, so a reader can walk from the report to the code and back.

The test file's own docstring states the rule: every “this should be quieter” assertion is paired with a “this must stay loud” one. A precision fix that is not paired with the true-positive it must preserve is not accepted. That is the mechanical form of the no-regression bar this project holds itself to.

WHAT IS NOT NEW, AND WHO SOLVED WHICH PART

Static analysis, secret detection, dependency scanning and attack-surface mapping are all solved problems with excellent open-source tools. Anyone claiming otherwise is selling something. This tool **uses** them and adds the a repository-aware planning layer. The benchmark protocol exists; no unrun head-to-head result is claimed.

The scanners

Semgrep, Trivy, Gitleaks, Checkov, OSV-Scanner, Prowler and the per-stack SAST tools: Bandit, gosec, Brakeman. Eleven registry entries, each with a minimum version that `doctor` checks, because presence on PATH is not compatibility. The tool shells out, parses, de-duplicates, and cites each finding back to the scanner that produced it.

Attack-surface detection

OWASP Noir extends route discovery beyond the fallback patterns. In the dated v0.14.0 proof the harness scores 10/10 with Noir and 8/10 without it. Both numbers are published, because a user without Noir deserves to know what they are missing.

Dynamic scanners

ZAP, Nuclei and sqlmap test a live system. The briefing suggests relevant signatures, not guaranteed detections. BOLA, missing auth, mass assignment and row-level-security gaps often need identity-aware manual controls. A clean DAST run is not “safe,” and the briefing says why.

WHAT IS DIFFERENT: THE QUESTION BEING ASKED

A SCANNER	WEBSEC-VALIDATOR
Matches patterns, reports all of them “Which lines look dangerous?” → 163 findings, one severity each A list. Every entry may be true. Nothing says which one a pentester would open first, what request would prove it, or which findings are the same bug seen from six files.	Assembles a pentester's handoff “What would I test first, against which endpoint, and what would confirm it?” → 13 issues, each with a location, a phase, an oracle, and a probe file A briefing. Phase 1 safe recon, Phase 2 authorization with two identities, Phase 3 injection fired only at sink-backed endpoints and gated behind an explicit warning.

ILLUSTRATIVE BRIEFING ITEM — THE SHAPE, NOT A REAL FINDING

\$5b · Phase 2 · authz · two identities

GET /api/orgs/{id}/invoices ← route with no per-object ownership check (authz_dataflow)

Oracle: tenant B reads tenant A's protected object, verified against owner and denied-access controls.

PROBES/BOLA-CROSS-TENANT.SH · STATUS ALONE IS INCONCLUSIVE · IDENTITY, OWNERSHIP AND CONTENT EVIDENCE REQUIRED

FIG. 8 — A FINDING KEEPS ITS NAME, ITS ENDPOINT AND ITS ORACLE. THAT IS THE DIFFERENCE BETWEEN A METRIC AND A PLAN.

WHAT IT DELIBERATELY IS NOT

- **Not a scanner.** It runs the ones you have installed and would be poorer without them.
- **Not a SaaS.** No hosted service, account or telemetry; zero runtime dependencies. Optional MCP HTTP is a token-protected loopback server.
- **Not a DAST.** Data probes default to GET; explicit TEST login may POST. Write probes need opt-in and a localhost target, including manually executed drafts.
- **Not a compliance tool.** No verdict, no score, no badge, and no signed attestation, because a websec-signed attestation would only attest that websec ran.
- **Not an autonomous fixer.** Core analysis never edits scanned source. Installing managed guidance or hooks is a separate explicit action.

ONE ENGINE, SEVEN SURFACES

SURFACE	WHAT IT GIVES YOU
CLI	<code>websec run .</code> is the whole pipeline. <code>gate</code> is the fast scoped pass on the files you just changed; <code>attest</code> projects a run into an audit-evidence table; <code>feedback</code> records that a finding is wrong, metadata-only by default, because a hardcoded-secret finding's snippet <i>is</i> the secret.
SARIF 2.1.0	Written on every run and importable back in, so GitHub code scanning, IDEs and other tools read the same ledger.
GitHub Action	Composite action with a <code>fail-on</code> threshold, a baseline for legacy repositories, and SARIF upload. Every third-party action it calls is pinned by commit SHA. It installs the engine from its own checkout, never from the target repository.
Docker	Everything bundled on a slim Python 3.14 image, running as a non-root user, with Noir 1.0.0, Gitleaks 8.30.1 and Trivy 0.74.0 pinned by version.
MCP server	Four read-only repository tools, <code>websec_recon</code> , <code>websec_findings</code> , <code>websec_sarif</code> and <code>websec_briefing</code> , plus advisory <code>websec_check_updates</code> , over stdio or loopback-only HTTP that requires a token. Release checks default to offline metadata; online checks require consent and never install packages. Non-loopback bindings are rejected before listening.
pre-commit	One hook id, defaulting to <code>--require-complete --fail-on high</code> , so a partial run cannot pass the commit.
Claude Code plugin	A security-pass skill and a <code>PostToolUse</code> hook: findings and incomplete analysis block; unexpected internal crashes fail open, loudly.

FOUR MONTHS, THIRTY-TWO RELEASES, ONE BREAKING CHANGE



FIG. 9 — SHIPPED HISTORY THROUGH 30 SEPTEMBER 2026. RELEASES DO NOT IMPLY RERUNNING THE DATED PROOF OR PUBLIC REVIEW.

CHECK, DO NOT UPDATE

Before review, the AI offers a release check and respects a decline. `websec update-check` is offline; `--online` requires consent. Only version/check-time metadata is cached; stale observations are dated. Checking never installs anything. An upgrade needs separate approval; refresh installed guidance explicitly afterward.

This is a reviewed v0.20.0 snapshot, not a claim of complete detection. Selected fact tests and the PDF freshness manifest guard publication; they do not prove every sentence. Dated review/proof artifacts retain their original revisions. If the code does not support a claim here, please open an issue.